

CKD Monitoring Data Warehouse

Digital Egypt Pioneers Initiative (DEPI) - Microsoft Data Engineering
SQL Server + Python ETL

1. Project Overview

This project builds a healthcare-focused Data Warehouse for monitoring Chronic Kidney Disease (CKD) progression over time. The solution integrates SQL Server (OLTP and DWH), a Python OOP ETL pipeline, and analytical SQL reports to enable trend analysis, early warning detection, and high-risk patient identification.

1.1 Key Outcomes

- Star schema Data Warehouse (CKD_DWH) with fact and dimension tables.
- Python ETL pipeline that extracts from CKD_OLTP, transforms clinical metrics, and loads into CKD_DWH.
- Computed clinical indicators: eGFR (CKD-EPI 2021), UACR (Albumin/Creatinine ratio), CKD staging (G1-G5) and albumin staging (A1-A3).
- Patient-level summary layer for fast, production-like reporting.
- SQL reports for progression, early warnings, and high-risk cohorts.

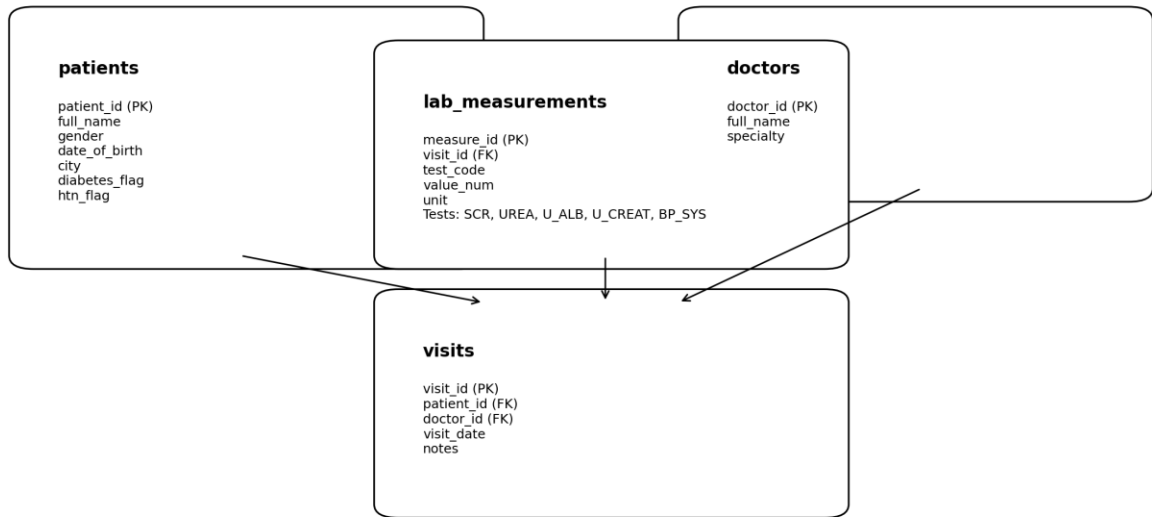
2. Architecture

High-level flow:

- Source OLTP database: CKD_OLTP (patients, visits, lab_measurements).
- ETL: Python (pyodbc) using OOP design.
- Target Data Warehouse: CKD_DWH (star schema + summary table).
- Reporting: SQL queries executed on CKD_DWH.

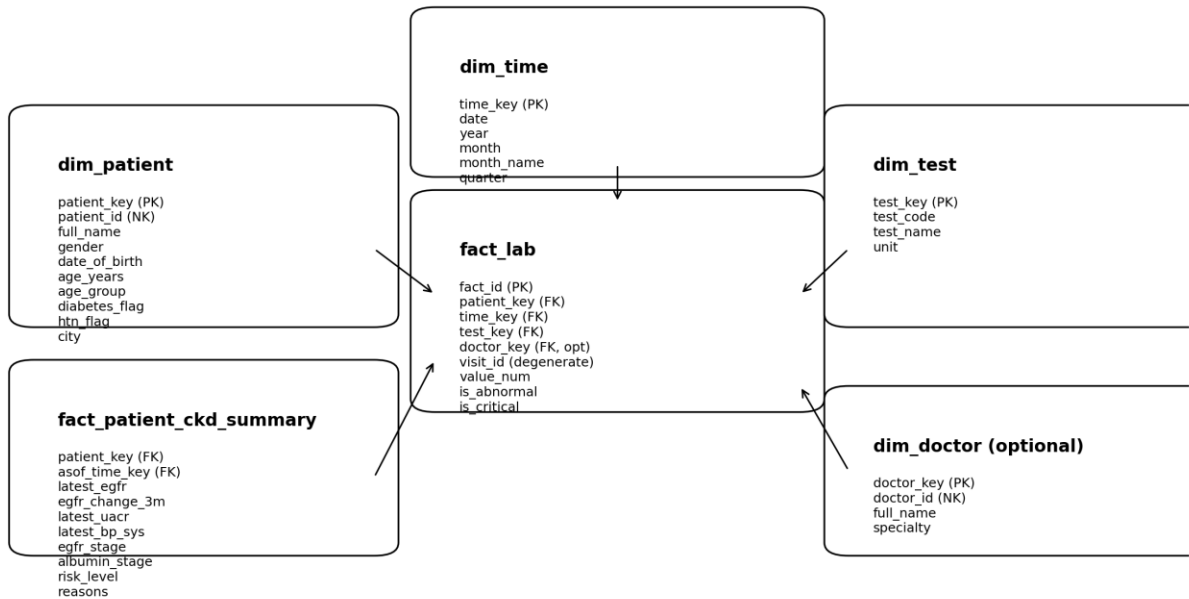
2.1 Diagram - OLTP ERD

OLTP ERD (CKD_OLTP)



2.2 Diagram - DWH Star Schema

Star Schema (CKD_DWH)



3. Data Model

3.1 OLTP (CKD_OLTP) Tables

- patients: demographic and risk factor flags (diabetes, hypertension).
- doctors: optional dimension for attribution.
- visits: patient encounter dates and notes.
- lab_measurements: lab and vital signs per visit.

Measurements used:

- SCR: Serum Creatinine (mg/dL)
- UREA: Urea (mg/dL)
- U_ALB: Urine Albumin (mg) and U_CREAT: Urine Creatinine (g) to compute UACR (mg/g)
- BP_SYS: Systolic blood pressure (mmHg)

3.2 Data Warehouse (CKD_DWH) Tables

- dim_patient: patient demographics and derived age_group.
- dim_time: date dimension (time_key = YYYYMMDD).
- dim_test: test catalog for SCR, UREA, UACR, BP_SYS, EGFR.
- dim_doctor (optional): doctor lookup.
- fact_lab: atomic measurements with abnormal and critical flags.
- fact_patient_ckd_summary: patient snapshot for reporting (latest values, changes, stages, risk).

4. Clinical Logic (Transform Layer)

4.1 eGFR (CKD-EPI 2021, no race)

eGFR is computed from Serum Creatinine, age, and gender using the CKD-EPI 2021 equation. This project implements the equation in Python during ETL, and stores the computed value as a fact measurement (test_code = EGFR).

4.2 UACR (Albumin/Creatinine Ratio)

UACR is computed as Urine Albumin (mg) divided by Urine Creatinine (g), resulting in mg/g. In OLTP, albumin and urine creatinine are stored as U_ALB and U_CREAT per visit. During ETL, UACR is computed and loaded as a fact measurement (test_code = UACR).

4.3 Staging and Risk

- eGFR staging: G1 (≥ 90), G2 (60-89), G3a (45-59), G3b (30-44), G4 (15-29), G5 (< 15).
- Albumin staging (UACR): A1 (< 30), A2 (30-300), A3 (> 300).
- Risk levels (demo rules): HIGH if eGFR < 30 , or (eGFR < 60 and UACR > 300), or rapid decline ≥ 10 in ~ 3 months; MED for eGFR < 60 or BP_SYS ≥ 140 ; otherwise LOW.

5. ETL Implementation

5.1 OOP Components

- DBConnector: manages SQL Server connections (Trusted or SQL login).
- Extractor: reads OLTP tables via SQL queries.
- Logic: transformation layer for keys, eGFR, UACR, staging, flags, and risk.
- Loader: upserts dimension tables, inserts fact rows, and rebuilds the summary snapshot.
- ETLPipeline: orchestrates Extract -> Transform -> Load.

5.2 Run Instructions

- Run SQL scripts in SSMS: 01_create_oltp.sql then 02_create_dwh.sql.
- Install dependency: pip install pyodbc
- Edit src/etl_pipeline.py to set your SQL Server instance (e.g., .\SQLEXPRESS).
- Run: python src/etl_pipeline.py
- Run reporting queries from 03_reports.sql.

6. Reporting

Core reports delivered (SQL):

- Progression over time: eGFR trend per patient.
- Early warning indicators: MED/HIGH risk snapshots with reasons.
- High-risk cohort: patients in HIGH risk category.
- Stage distribution: counts by (G stage, A stage).
- Monthly monitoring volume: visits per month.

7. Future Enhancements

- Power BI dashboard: KPIs, trends, and patient drill-down pages.
- Azure deployment: Azure SQL Database + orchestration (Azure Data Factory) + monitoring.
- Machine learning (future phase): predict risk escalation or eGFR decline based on longitudinal features.

Appendix A. GitHub Repository Structure

Recommended GitHub repo structure:

```
ckd-monitoring-dwh/  
├─ README.md  
├─ sql/  
│   └─ 01_create_oltp.sql
```

```
|  ├── 02_create_dwh.sql
|  └── 03_reports.sql
├── src/
|   ├── etl_pipeline.py
├── diagrams/
|   ├── ERD_OLTP.png
|   └── StarSchema_DWH.png
└── docs/
    └── CKD_Monitoring_Data_Warehouse_Documentation.docx
```